

第二阶段阶段性训练与分场景评估总结

更新时间：2026-07-08

1. 本轮目标

本轮目标不是直接把所有信号类型一次性强行训练到最好，而是按稳定性逐步推进：

- 1. 先从较简单、较稳定的信号开始，例如线性 chirp、二次多项式 chirp、三次多项式 chirp。
- 2. 如果简单场景下 IF-Net + 可微 ICCD 层能够稳定重构，再扩展到全部类型。
- 3. 对全部类型做分场景评估，找出哪些类型已经满足第二阶段继续推进的需要，哪些类型还需要单独优化。
- 4. 对困难类型尝试专家模型或专门配置，判断是否能通过更长训练解决。

2. 已完成的代码工作

本轮在第二阶段代码中补充了分场景评估和多种训练配置：

- 新增 `src/stage2_iccd/eval_scenarios.py`，用于对指定 checkpoint 按信号类型分别评估。
- 新增 `configs/separated_frozen.yaml`，用于简单分离场景的课程训练。
- 新增 `configs/all_multiexpert.yaml`，用于多 IF-Net 专家候选输入的全类型训练实验。
- 新增 `configs/local_jump_frozen.yaml`，用于 `local_jump` 专门训练。
- 新增 `configs/sinusoidal_frozen.yaml`，用于 `sinusoidal_fm` 专门训练。
- 修改 `model.py` 中的候选 IF 混合器：
- 原先是全局可学习平均，容易把不同专家输出的 IF 曲线直接平均成不存在的轨迹。
- 现在增加基于 ICCD 初步重构残差的动态候选权重，让每个样本根据重构误差选择更合适的候选。
- 修改训练日志，记录候选权重均值和候选选择温度。

3. 训练路径与主要结果

3.1 简单分离场景训练

训练对象：linear、quadratic、cubic。

使用配置：`configs/separated_frozen.yaml`

训练后 checkpoint：`stage2_iccd/runs/separated_frozen/latest.pt`

分场景评估结果：

场景	重构 SNR / dB	IF MAE / Hz
linear	23.40	2.38
quadratic	23.02	2.47
cubic	21.64	3.55
aggregate	22.69	2.80

结论：简单场景训练是稳定的。说明第二阶段的可微 ICCD 层、alpha 学习、候选 IF 权重和小型 IF refinement head 都能正常参与训练，并且没有出现明显梯度或重构不稳定问题。

3.2 从简单场景扩展到全部类型

使用 `separated_frozen` 作为初始化，继续训练全部类型。

训练后 checkpoint：`stage2_iccd/runs/all_from_separated/latest.pt`

这是目前最稳的通用第二阶段 checkpoint。

分场景评估结果：

场景	重构 SNR / dB	IF MAE / Hz	判断
linear	21.18	3.26	可接受
quadratic	21.76	2.40	较好
cubic	21.52	3.22	可接受
sinusoidal_fm	17.67	5.58	偏弱
crossing	19.97	3.03	可接受
near_parallel	23.35	1.71	最稳定
local_jump	16.17	6.70	明显偏弱
tangent_or_overlap	21.11	3.13	可接受
aggregate	20.34	3.63	总体可用

结论：全类型一起推进是可行的，但不是所有类型同步受益。当前通用模型对 linear、quadratic、cubic、crossing、near_parallel、tangent_or_overlap 的表现已经比较稳定；主要短板集中在 local_jump 和 sinusoidal_fm。

4. 困难类型的补充实验

4.1 sinusoidal_fm 专门模型

训练后 checkpoint：`stage2_iccd/runs/sinusoidal_frozen/latest.pt`

评估结果：

场景	重构 SNR / dB	IF MAE / Hz
sinusoidal_fm	20.28	4.05

相对通用模型：

- 通用模型：17.67 dB, 5.58 Hz
- sinusoidal 专门模型：20.28 dB, 4.05 Hz

结论：sinusoidal_fm 可以通过专门训练明显改善。后续可以保留通用模型，也可以在路由器足够稳定后为 sinusoidal_fm 使用专门分支。

4.2 local_jump 专门模型

训练后 checkpoint：`stage2_iccd/runs/local_jump_frozen/latest.pt`

评估结果：

场景	重构 SNR / dB	IF MAE / Hz
local_jump	15.15	8.34

相对通用模型：

- 通用模型：16.17 dB, 6.70 Hz
- local_jump 专门模型：15.15 dB, 8.34 Hz

结论：local_jump 不是简单换成 local_jump IF-Net checkpoint 或继续训练 ICCD 层就能解决。它的误差主要来自跳变位置和跳变前后 IF 段的结构没有被第二阶段显式建模。当前的 ICCD refinement head 更擅长连续、小幅、平滑修正，对突变点附近的非平滑 IF 变化表达能力不足。

5. 多专家候选混合实验

本轮尝试了多 IF-Net 专家候选输入，包括 balanced、polynomial、sinusoidal、local_jump 等多个第一阶段 checkpoint。

5.1 全局平均式候选混合

结果较差：

- validation 重构 SNR 约 13.45 dB
- validation IF MAE 约 14.40 Hz

主要原因：不同专家输出的 IF 曲线不一定是同一个物理分量的同一个候选。直接做全局可学习平均时，可能把几条合理曲线平均成一条不真实的中间曲线，尤其在 crossing、local_jump、相切或短时重合场景中更明显。

5.2 基于 ICCD 残差的动态候选混合

动态混合后，模型会根据每个候选 IF 的初步 ICCD 重构残差来给权重。

结果仍不适合作为主路径：

- aggregate 重构 SNR 约 16.54 dB
- aggregate IF MAE 约 11.45 Hz
- crossing 出现明显退化，重构 SNR 约 4.32 dB, IF MAE 约 30.43 Hz

原因：ICCD 残差本身并不总能区分“物理身份正确的 IF”和“短时间内也能解释能量但身份错误的 IF”。在 crossing 场景中，候选曲线可能局部重构误差不大，但分量身份已经交换，导致后续 ICCD 展开层沿错误轨迹分解。

结论：多专家不是不能用，但不能只靠残差选择。后续如果使用多专家，需要加入场景判别器或身份一致性约束，让路由器先判断信号结构，再决定使用哪个专家或怎样融合 top-k 候选。

6. 当前最佳使用建议

当前第二阶段最稳组合：

- 通用模型：stage2_iccd/runs/all_from_separated/latest.pt
- sinusoidal_fm 专门模型：stage2_iccd/runs/sinusoidal_frozen/latest.pt

建议：

1. 一般场景优先使用 `all_from_separated`。
2. 如果前端判别器较确定是 `sinusoidal_fm`，可以切换到 `sinusoidal_frozen`。
3. 暂时不要把 `all_multiexpert_dynamic` 作为主模型，因为它在 `crossing` 上不稳定。
4. `local_jump` 暂时继续使用通用模型，直到第二阶段加入显式跳变位置建模。

7. 对第二阶段任务的影响

第二阶段的核心目标是用可微 ICCD 展开层把第一阶段的 IF 初始估计转化为可重构、可反传、可联合优化的分量分解过程。

从当前结果看：

- 简单场景和多数连续 IF 场景已经能支撑第二阶段继续推进。
- `crossing` 当前通用模型表现可接受，但多专家路由会引入身份交换风险，因此后续必须保留 top-2 候选和身份一致性检查。
- `local_jump` 会影响第二阶段的端到端精修，因为跳变点附近 IF 误差会直接导致相位积分误差，进而影响 ICCD 字典构造和分量重构。
- `sinusoidal_fm` 需要专门路由或更强的周期性 IF refinement，但问题比 `local_jump` 更容易通过专门模型缓解。

8. 下一步优化方向

优先级建议如下：

1. 为 `local_jump` 增加跳变位置辅助输入或辅助头。
 - 第一阶段已经有跳变事件定位思路，但第二阶段目前没有显式使用它。
 - 应把 `jump position`、`jump confidence` 或 `jump mask` 输入到 `refinement head`。
 - `refinement head` 在跳变点附近允许更大、更局部的 IF 修正。
2. 把 ICCD 展开层改成局部/分段式 IF refinement。
 - 对 `local_jump`，不能只用全局平滑修正。
 - 可以在跳变点前后分别建模 IF 曲线或分别设置正则强度。
3. 为 `crossing` 加身份一致性约束。
 - 保留 top-2 候选。
 - 在时间轴上限制分量身份频繁交换。
 - 训练损失中加入轨迹连续性或最优匹配约束。
4. 重新设计多专家路由器。
 - 不建议只用 ICCD 残差。
 - 应结合信号类型判别器、候选置信度、身份一致性和局部重构误差。
5. 在通用模型稳定后，再逐步解冻 IF-Net。
 - 目前仍建议先固定第一阶段 IF-Net，只训练 `alpha`、候选权重和 `refinement head`。
 - 等 `local_jump` 和 `crossing` 的第二阶段机制补足后，再做端到端联合训练。

9. 当前是否可以继续第二阶段

可以继续第二阶段，但需要分层推进：

- 对 linear、quadratic、cubic、near_parallel、tangent_or_overlap、crossing：可以继续使用当前通用模型推进可微 ICCD 展开层和端到端训练框架。
- 对 sinusoidal_fm：可以使用专门模型作为临时分支，并继续优化路由。
- 对 local_jump：不建议直接进入大规模联合训练，应先补充跳变位置辅助头或分段式 refinement，否则端到端训练可能把误差传回 IF-Net，造成不稳定的错误修正。

因此，当前判断是：第二阶段框架已经成立，通用模型基本可用；但 local_jump 需要作为下一轮重点结构优化，而不是只依赖更长训练。

10. 2026-07-08 补充：先打牢简单多分量

根据“先专攻单分量与简单多分量，交叉、跳变等困难类型后续再攻克”的路线，本轮先没有把 crossing、local_jump 等困难类型继续混入训练，而是集中处理 linear、quadratic、cubic 这三类简单分离多分量信号。

10.1 为什么暂时没有直接做真正单分量

当前第二阶段使用的冻结 IF-Net checkpoint 是按 2 分量输出训练的。如果直接把真实 1 分量样本喂给第二阶段，第二条不存在的分量会带来两个问题：

- component loss 可以用零分量匹配处理，但 IF loss 仍会惩罚第二条“并不存在”的 IF；
- 可微 ICCD 层会尝试解释不存在的第二分量，可能把噪声当作弱分量重构。

因此，真正单分量需要先加入 inactive component mask 或 active-component loss mask。为了保证优化路径干净，本轮先做“简单、分离、2 分量”。

10.2 simple_multicomponent_long

配置：configs/simple_multicomponent_long.yaml

训练路径：

- 从 stage2_iccd/runs/separated_frozen/latest.pt 续训；
- 只使用 linear、quadratic、cubic；
- 训练噪声先限制为 white + colored；
- SNR 范围设为 4 dB 到 28 dB；
- 继续训练 800 步。

训练后 checkpoint：stage2_iccd/runs/simple_multicomponent_long/latest.pt

与原 separated_frozen 在同一 easy 条件下比较：

模型	aggregate SNR / dB	aggregate IF MAE / Hz	smooth
separated_frozen	25.62	2.20	9.21
simple_multicomponent_long	25.93	1.64	2.59

分场景结果：

场景	SNR / dB	IF MAE / Hz
linear	26.74	1.30
quadratic	25.94	1.58
cubic	25.12	2.06
aggregate	25.93	1.64

结论：这一步达到了较好的效果。IF 精度明显提升，曲线二阶平滑度明显降低，说明第二阶段 refinement head 没有只是追求重构误差，而是把 IF 轨迹也修得更稳。

10.3 鲁棒条件复测

在更复杂条件下评估：

- SNR 范围：-2 dB 到 24 dB；
- 噪声：white 0.55、colored 0.25、impulsive 0.10、trend 0.10。

模型	aggregate SNR / dB	aggregate IF MAE / Hz	smooth
separated_frozen	22.94	2.91	13.68
simple_multicomponent_long	22.86	2.55	4.46

结论：鲁棒条件下重构 SNR 基本持平，IF MAE 和 smoothness 仍然更好。因此 simple_multicomponent_long 可以作为当前简单多分量阶段的最佳模型。

10.4 simple_multicomponent_robust 尝试

配置：configs/simple_multicomponent_robust.yaml

训练路径：

- 从 simple_multicomponent_long/latest.pt 续训；
- 训练时加入 impulsive 和 trend 噪声；
- 继续训练 600 步。

结果：

评估条件	SNR / dB	IF MAE / Hz	smooth
easy	25.53	1.74	2.21
robust	22.48	2.58	3.23

结论：鲁棒续训没有超过 simple_multicomponent_long。它让 IF 曲线更平滑，但重构 SNR 略降，且 robust IF MAE 没有实质改善。因此它目前只作为诊断实验保留，不作为第一步最佳 checkpoint。

10.5 当前第一步判断

当前简单多分量阶段可以视为初步达标：

- easy 条件 aggregate IF MAE 已降到约 1.64 Hz；
- robust 条件 aggregate IF MAE 约 2.55 Hz；
- linear/quadratic/cubic 都没有明显失控；
- 可微 ICCD 层的 alpha、候选权重和 IF refinement head 都能稳定训练。

下一步不建议立刻攻 local_jump。更稳的顺序是：

1. 增加 active-component mask，补真正单分量训练和评估；
2. 在简单多分量基础上加入 near_parallel 或更小间隔的非交叉多分量；
3. 简单和近邻场景都稳定后，再进入 crossing；
4. 最后单独攻 local_jump，并加入跳变位置辅助头。

11. 2026-07-08 补充：单分量与单/多分量合并尝试

11.1 active-component mask

为了正确处理真正单分量，代码中新增了 active-component mask：

- 仿真器现在可以通过 active_components 指定真实活跃分量数；
- inactive component 的幅值设为 0；
- component/reconstruction loss 仍约束模型不要重构不存在的分量；
- IF loss 只在 active component 上计算，避免用不存在的第二条 IF 污染训练。

这一步是必要的。否则单分量训练会把“第二条不存在的 IF”当作真实目标，造成错误优化。

11.2 单分量专用模型

配置：configs/simple_single_component.yaml

训练路径：

- 从 simple_multicomponent_long/latest.pt 续训；
- 只使用 linear、quadratic、cubic；
- active_components: 1；
- 训练 500 步。

checkpoint：stage2_iccd/runs/simple_single_component/latest.pt

单分量 easy 条件评估：

场景	SNR / dB	IF MAE / Hz	component L1
linear	26.64	0.65	0.029
quadratic	27.71	0.69	0.025
cubic	26.83	0.73	0.028
aggregate	27.06	0.69	0.028

单分量 robust 条件评估：

场景	SNR / dB	IF MAE / Hz	component L1
linear	23.35	0.73	0.041
quadratic	23.87	0.78	0.039
cubic	23.71	0.82	0.040
aggregate	23.64	0.78	0.040

结论：单分量专用模型效果很好，说明 active-component mask 是有效的。

11.3 单分量模型不能直接处理双分量

把 `simple_single_component/latest.pt` 放到 2 active components 的 easy 条件下评估：

条件	SNR / dB	IF MAE / Hz	component L1
2 分量 easy	7.57	7.07	0.223

结论：单分量模型非常专用，不能直接用于双分量。它会倾向于只解释一个主分量，导致另一个真实分量重构失败。

11.4 双分量模型处理单分量的表现

把 `simple_multicomponent_long/latest.pt` 放到 1 active component 的 easy 条件下评估：

条件	SNR / dB	IF MAE / Hz	component L1
1 分量 easy	28.49	1.32	0.189

结论：双分量模型对单分量的重构 SNR 和 IF 还可以，但 component L1 很高，说明它会把单个真实分量拆到两个输出槽位中。这对后续可解释分量分解是不利的。

11.5 mixed-active 统一模型尝试

配置：`configs/simple_active_mixed.yaml`

训练路径：

- 从 `simple_multicomponent_long/latest.pt` 续训；
- 训练样本中 45% 为单分量，55% 为双分量；
- 训练 700 步。

评估结果：

条件	SNR / dB	IF MAE / Hz	component L1
1 分量 easy	27.58	1.53	0.063
2 分量 easy	23.71	2.96	0.061
mixed robust	22.71	2.70	0.070

结论：mixed-active 统一模型没有超过专用模型组合。它比单分量专用模型差，也比双分量专用模型差，说明单/多分量在当前第二阶段结构中存在明显任务冲突。

11.6 当前最稳组合

当前不建议强行用一个模型同时处理单分量和双分量。最稳组合是：

- 单分量：`stage2_iccd/runs/simple_single_component/latest.pt`
- 简单双分量：`stage2_iccd/runs/simple_multicomponent_long/latest.pt`

进入下一步之前，建议先加 active-component 判别器或能量型路由器：

- 若判定为 1 active component，走单分量专用模型；
- 若判定为 2 active components，走简单双分量模型；

- 如果判别置信度低，保留 top-2 active-count 候选，进入第二阶段时同时输出置信度。

因此，当前简单单分量和简单双分量都已经有了可用模型，但还需要一个轻量 active-count 路由器，才能形成完整稳定的第一层处理流程。

12. 2026-07-08 补充：active-count 路由器与 near_parallel 纳入

在完成单分量专用模型和简单双分量专用模型之后，本轮继续补上了一个轻量 active-count 路由器。它的任务不是直接预测完整 IF，而是先判断输入信号更像 1 个活跃分量还是 2 个活跃分量，然后把样本送到对应的第二阶段模型：

- 1 active component：使用 `stage2_iccd/runs/simple_single_component/latest.pt`；
- 2 active components：使用 `stage2_iccd/runs/simple_multicomponent_long/latest.pt`。

这样做的原因是，前面的实验已经说明“一个统一模型同时处理单分量和双分量”会出现明显任务冲突。单分量模型很擅长单分量，但不能直接处理双分量；双分量模型能重构单分量，却容易把一个真实分量拆到两个输出槽位里。因此先做 active-count 判别，再走专门模型，比强行用一个模型覆盖所有情况更稳。

12.1 新增代码

本轮新增了以下代码和配置：

- `src/stage2_iccd/active_count.py`：active-count 分类网络。输入来自多尺度 STFT 图和辅助统计特征，输出 `active_1/active_2` 两类概率、置信度和 margin；
- `src/stage2_iccd/train_active_count.py`：active-count 路由器训练脚本；
- `src/stage2_iccd/eval_active_count.py`：active-count 单独评估脚本；
- `src/stage2_iccd/eval_active_routed_stage2.py`：把 active-count 路由器接到单分量/双分量 Stage2 模型后的端到端评估脚本；
- `configs/active_count_simple.yaml`：只包含 linear、quadratic、cubic 的初始路由器配置；
- `configs/active_count_simple_near_parallel.yaml`：在简单场景基础上加入 near_parallel 的路由器配置。

12.2 初始 active-count 路由器

先用 `active_count_simple.yaml` 训练，只覆盖 linear、quadratic、cubic 的 1/2 active component 判别。

在简单 easy 条件下：

评估范围	accuracy	active_1 accuracy	active_2 accuracy
linear/quadratic/cubic	99.1%	99.7%	98.5%

在 robust 条件下：

评估范围	accuracy	active_1 accuracy	active_2 accuracy
linear/quadratic/cubic	97.3%	98.1%	96.5%

接到 Stage2 后，linear/quadratic/cubic 的 routed 结果为：

条件	route accuracy	IF MAE / Hz	重构 SNR / dB
easy	99.4%	1.30	26.01
robust	97.7%	1.85	23.09

结论：只看简单场景时，active-count 路由器已经足够稳定，可以把单分量和双分量模型组合起来使用。

12.3 near_parallel 的问题与修正

继续把同一个 active-count 路由器放到 near_parallel 上评估，发现准确率只有约 90%：

条件	accuracy	active_1 accuracy	active_2 accuracy
near_parallel easy	90.3%	85.6%	95.0%
near_parallel robust	90.2%	88.9%	91.6%

这个结果说明 near_parallel 的主要问题不在第二阶段 ICCD 重构本身，而在“路由器没有见过这类结构”。near_parallel 的时频图中，两条 IF 长时间接近并近似平行，能量分布容易被误判成一个宽一些的单分量脊线；反过来，某些单分量的缓慢弯曲也可能被误判成两个贴近分量。因此如果 active-count 路由器只用普通 separated linear/quadratic/cubic 训练，面对 near_parallel 会不稳定。

为了修正这个问题，本轮训练了 active_count_simple_near_parallel.yaml，把 near_parallel 一并加入 active-count 训练。

修正后，near_parallel 路由准确率明显提高：

条件	accuracy	active_1 accuracy	active_2 accuracy
near_parallel easy	99.1%	100.0%	98.3%
near_parallel robust	97.6%	99.8%	95.3%

同时重新检查 linear/quadratic/cubic，没有发生严重遗忘：

条件	accuracy	active_1 accuracy	active_2 accuracy
simple easy	98.6%	100.0%	97.2%
simple robust	96.8%	100.0%	93.7%

这里要注意一个小代价：加入 near_parallel 后，强噪声下简单双分量的路由准确率从约 96.5% 降到约 93.7%。这说明路由器容量和训练分布仍然有限，但端到端 Stage2 指标仍然可用。

12.4 新路由器接入 Stage2 后的结果

使用新路由器 active_count_simple_near_parallel/latest.pt，并接入：

- 单分量模型：simple_single_component/latest.pt；
- 双分量模型：simple_multicomponent_long/latest.pt。

在 linear、quadratic、cubic、near_parallel 四类上做 routed Stage2 评估，结果如下：

条件	route accuracy	IF MAE / Hz	重构 SNR / dB	component L1
easy	98.8%	1.18	26.21	0.039
robust	97.3%	1.64	23.42	0.050

分场景看，单分量 IF MAE 基本稳定在 0.62-0.86 Hz；双分量中 near_parallel 的 IF MAE 为 easy 1.30 Hz、robust 1.66 Hz，说明第二阶段 ICCD 对 near_parallel 本身并不弱。当前 near_parallel 的主要风险已经从“重构能力不足”转移到“路由器是否足够稳定”。

12.5 当前判断

到目前为止，第一批可继续推进的第二阶段场景包括：

- 单分量 linear / quadratic / cubic ；
- 简单双分量 linear / quadratic / cubic ；
- near_parallel。

这些场景已经满足继续推进的基本门槛：路由准确率在 easy/robust 条件下都接近或超过 97%，端到端 IF MAE 在 1-2 Hz 左右，重构 SNR 大约 23-26 dB。

暂时不建议直接把 crossing 和 local_jump 加进同一个训练任务。原因是：

- crossing 的主要风险是分量身份交换，不能只靠 active-count 路由解决；
- local_jump 的主要风险是跳变位置定位和跳变点附近的分段 IF 修正，需要显式 jump head 或 jump mask ；
- sinusoidal_fm 比 crossing/local_jump 更适合作为下一步，因为它仍属于连续 IF，只是周期性调制更强，适合作为从简单连续场景到困难场景之间的过渡。

12.6 下一步优化顺序

下一轮建议按下面顺序推进：

1. 把 sinusoidal_fm 纳入当前 active-count + routed Stage2 流程，先看路由是否稳定，再看 Stage2 是否需要专门分支；
2. 如果 sinusoidal_fm 的路由稳定但 IF MAE 偏高，优先训练 sinusoidal_fm 的 Stage2 分支，而不是马上解冻 IF-Net ；
3. 再进入 crossing，重点加入身份一致性约束和 top-2 候选保留；
4. 最后集中处理 local_jump，加入跳变位置辅助头、jump mask 或分段 refinement。

因此，本轮结论是：简单单/双分量和 near_parallel 的 Stage2 路由框架已经基本打通，可以继续往 sinusoidal_fm 扩展；但 crossing/local_jump 仍应作为后续专项攻克对象。